

Radar Project Progress Report

Timofey Kreslo, Thomas Kemper, Usejd Aliu

April 18, 2025

1 Achievements

We successfully captured and visualized raw ADC data from the AWR1843BOOST mmWave radar using the DCA1000 board. Initially, we used offline processing with range FFT and 2D beamforming on fixed-frame recordings, which yielded promising visualizations of human targets — especially in uncluttered environments. We've included some examples in the following Drive folder: [drive link](#). As shown, human targets are clearly visible when the scene is free of objects. However, the presence of objects, particularly metal ones, introduces significant noise, making it harder to accurately distinguish people.

We also managed to run the algorithm in real-time. Although it works, the latency is high, and the same noise issue persists in cluttered environments.

We also implemented DBSCAN clustering on filtered heatmaps, achieving good separation between individuals, even with noisy input. The real-time system now effectively visualizes and counts people based on movement and beamformed signals. Here is an example with two people in the scene, where we apply the following pipeline: beamforming $\rightarrow$ thresholding to retain only strong signals $\rightarrow$ clustering using DBSCAN:

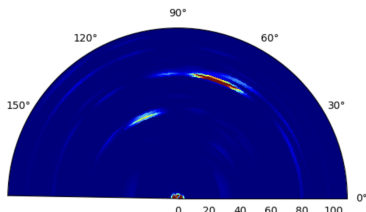


figure 1:2 people

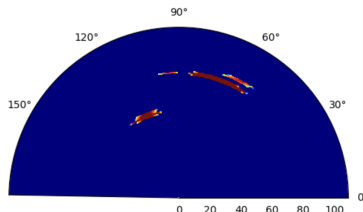


figure 2:add threshold

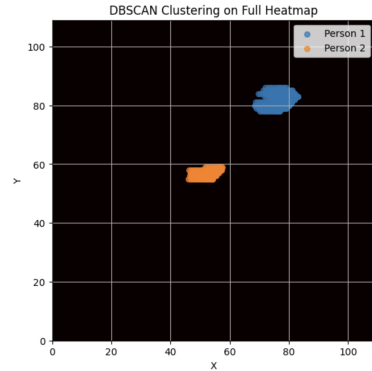


figure 3:DBSCAN

2 Status and Deviations from Proposal

As far as our proposal is concerned, for the first stage, we successfully captured and pre-processed the radar data as planned. We have implemented the range FFT, and the Doppler FFT is scheduled for the coming weeks.

For the second stage, we generated a 2D heatmap. So far, we've worked with a single radar, focusing on optimizing performance before scaling up to multiple units. The confidence map has not been implemented, as we've made some revisions to our initial plan.

Specifically, we've decided to treat Step 3 (Deep Learning-Based Shape Reconstruction) as an optional feature rather than part of the project's core functionality. Hence, in our updated plan, Step 3 becomes Step 4 : Real-Time Optimization (priority task), and the original Step 4 is moved up to Step 3. This change allows us to concentrate our efforts on developing real-time capabilities, which we consider more critical for the final product.

Our new objective for the final project is to generate a top-view heatmap that can accurately differentiate and count people in real time, with acceptable latency and frame rate.

3 Problems Encountered & Solution

Initially, we attempted to average 500 frames post-capture to reduce background noise. While this worked well, the computational cost was excessive (500×500 iterations), making it unsuitable for real-time applications.

To enable real-time processing, we shifted focus to efficient filtering methods. We implemented Gaussian and median filters to suppress noise effectively without compromising frame rate. This significantly improved visualization consistency and allowed us to observe human motion as 2D polar-projected "blobs" corresponding to actual movement.

One major issue was latency: 2D beamforming, while accurate, introduced processing delays due to its high computational load. This caused the queue to accumulate frames faster than they could be processed. To solve this, we redesigned the data pipeline to prioritize recency — discarding older frames and keeping only the latest for rendering. This solution allowed for approximately 3 FPS, which proved sufficient for tracking multiple individuals in real time, and we plan to adapt existing work (e.g., from the TU Delft thesis) to improve the signal pipeline in low-complexity ways.

4 Next steps

Over the next few weeks, our priority is to reduce noise and improve overall performance. To achieve this, we plan to draw inspiration from the pipeline presented in the thesis [People Counting Using Low-cost FMCW MIMO Radar](#). This thesis aligns closely with our current project and proposes two algorithms for pre-processing (in section 4):

Algorithm 2 4D Radar Cube Generation.

Require: $X_r(n_{Rx}, n_s)$, chirps per frame n_c , samples per chirp n_{adc} , total number of samples of each receiver n_s , the number of transmitters n_{Tx} , the number of receivers n_{Rx}

- 1: Calculate the number of frames $n_f = n_s / (n_{Tx} \times n_c \times n_{adc})$
 - 2: Calculate the number of virtual arrays $n_v = n_{Tx} \times n_{Rx}$
 - 3: Calculate the Read Cube $R_e = \text{linspace}(1, n_s, n_s)$
 - 4: Reshape the Read Cube to capture the samples generated by chirps of each Tx $R_e(n_s / (n_{adc} \times n_{adc}))$
 - 5: Use FOR loop to extract the samples transmitted by each Tx, and build the virtual array
 - 6: **for each** $i \in [1, n_t, (n_s / n_{adc})]$ **do**
 - 7: $X_R(n_a, n_{adc}, n_c, n_f) = [X_r(1, n_{Rx}, R_e(i), n_f), \dots, X_r(\max(n_a), n_{Rx}, R_e(i), n_f)]$
 - 8: **end for**
- Ensure:** $X_R(n_a, n_{adc}, n_c, n_f)$
-

figure 1: algo. 2

Algorithm 3 Pre-processing steps of detection and estimation in range, Doppler, angle.

Require: $X_R(n_a, n_{adc}, n_c, n_f)$

- 1: **for** $n = 1, 2, 3, \dots, n_f$ **do**
 - 2: Range and Doppler estimation based on 2D-FFT $X_R(n_a, n_R, n_D, i) = \text{FFT2}(X_R(n_a, n_{adc}, n_c, i))$
 - 3: Apply detection method CA-CFAR to determine the Range and Doppler bin for angle estimation $(n_R, n_D, i) = \text{CA-CFAR}(X_R(n_a, n_R, n_D, i))$
 - 4: Apply angle estimation FFT beamforming at the selected range-Doppler bins along Channel axis $n_A = \text{argmax}_A(\text{FFT}(X_R(n_a, n_R, n_D, i)))$
 - 5: **end for**
- Ensure:** $X_R(n_A, n_R, n_D, n_f)$
-

figure 2: algo. 3

These two algorithms offer the advantage of maintaining a certain level of simplicity while integrating seamlessly with what we've implemented so far. They are expected not only to reduce noise but also to significantly enhance the performance of our system by applying beamforming only to selected areas. For these reasons, we plan to integrate them into our project.

By reducing noise, we aim to obtain higher-quality data, which will improve the effectiveness of DB-SCAN (as shown in our earlier example) and enable more accurate people counting.

Once we've achieved strong performance with a single radar, we will move on to integrating additional radars for data capture. Depending on the progress of these two steps, we will then begin implementing the extra features:

- Integrate confidence maps for better detection certainty.
- Explore basic deep learning models for improved shape estimation.
- If time permits, investigate breathing analysis using micro-motion detection.